PROJECT TITLE: RETAIL MICROSERVICES SYSTEM

Business Requirements Document (BRD) for Retail Application

Table of Contents

1. Introduction Page 1
2. Objectives Page 1
3. Scope Page 1
4. Use Cases Page 1 - 4
5. Implementation Steps & Matrix Page 4
6. Non-Functional Requirements Page 5 - 6
7. Conclusion Page 7

1. Introduction

The Retail Microservices System aims to enhance retail operations by implementing a microservices architecture. This approach enables flexibility, scalability, and improved customer experience while streamlining various processes within the retail domain.

2. Objectives

- Improve customer engagement and satisfaction.
- Streamline inventory and order management.
- Enable rapid deployment of new features and updates.
- Ensure robust security and compliance with industry standards.

3. Scope

This document outlines key use cases relevant to the retail domain that will be implemented using a microservices architecture. Each use case includes detailed descriptions and workflows.

4. Use Cases

Use Case 1: Customer Registration

- **Description:** Customers can create an account on the retail platform.
- **Actors:** Customer, Registration Service
- **Preconditions:** The customer provides valid personal information.
- **Postconditions:** A new customer account is created, and the customer receives confirmation.
- **Main Flow:** Customer accesses the registration page via the web or mobile app, fills out the form with required information, validates information, creates a new customer record, and sends a confirmation email.

Use Case 2: Product Search

- **Description:** Customers can search for products available in the store.
- **Actors:** Customer, Product Service
- **Preconditions:** The customer must be logged in or browsing as a guest.
- **Postconditions:** The customer receives a list of matching products.
- **Main Flow:** Customer enters a search term in the search bar; Product Service retrieves products matching criteria; service formats product list including images and prices; customer views search results and applies filters to refine search.

Use Case 3: Shopping Cart Management

- **Description:** Customers can add, update, or remove items from their shopping cart.
- **Actors:** Customer, Shopping Cart Service
- **Preconditions:** The customer must be logged in or browsing as a guest.
- **Postconditions:** The shopping cart is updated with the selected items.
- **Main Flow:** Customer views a product detail page and selects "Add to Cart"; Shopping Cart Service adds item to cart in the database; customer views shopping cart and all added items; customer updates quantities or removes items; service updates cart and reflects changes on screen.

Use Case 4: Order Placement

- **Description:** Customers can place an order for items in their shopping cart.
- **Actors:** Customer, Order Service, Payment Service
- **Preconditions:** The customer must have items in their shopping cart.
- **Postconditions:** An order is created, and the customer receives confirmation.
- **Main Flow:** Customer navigates to checkout page from shopping cart; Order Service retrieves items from cart; customer enters shipping and billing information; Payment Service processes payment information; if successful, Order Service creates a new order and sends confirmation.

Use Case 5: Order Tracking

- **Description:** Customers can track the status of their orders.
- **Actors:** Customer, Order Service
- **Preconditions:** The customer must have placed an order.
- **Postconditions:** The customer views the current status of their order.
- **Main Flow:** Customer logs into account and navigates to "Order History"; Order Service retrieves list of past orders; customer selects an order to view details; service retrieves current status and tracking information from database; customer views order status and tracking details on screen.

Use Case 6: Product Review and Ratings

- **Description:** Customers can leave reviews and ratings for purchased products.
- **Actors:** Customer, Review Service
- **Preconditions:** The customer must have purchased the product.
- **Postconditions:** The review and rating are saved and visible to other customers.
- **Main Flow:** Customer navigates to product detail page for an item they purchased; selects "Write a Review"; Review Service prompts customer to enter rating and review text; service validates input and saves review in database; review is displayed on product page.

Use Case 7: Inventory Management

- **Description:** Store managers can manage product inventory levels.
- **Actors:** Store Manager, Inventory Service
- **Preconditions:** The manager must be authenticated and have necessary permissions.
- **Postconditions:** Inventory levels are updated in the system.
- **Main Flow:** Store Manager logs into inventory management portal; Inventory Service retrieves current inventory levels from database; manager selects a product to update inventory; manager enters new stock quantity and submits changes; service updates inventory levels in database and confirms update.

Use Case 8: Promotional Campaigns

- **Description:** Retailers can create and manage promotional campaigns for products.
- **Actors:** Marketing Manager, Campaign Service
- **Preconditions:** The manager must be authenticated and have necessary permissions.
- **Postconditions:** The promotional campaign is active and applied to relevant products.
- **Main Flow:** Marketing Manager logs into campaign management portal; Campaign Service allows manager to create new campaign; manager enters campaign details (discount rate, duration, targeted products); service saves campaign and activates it based on schedule; customers view promotional pricing on relevant product pages.

Use Case 9: Customer Support

- **Description:** Customers can submit support requests for assistance.
- **Actors:** Customer, Support Service
- **Preconditions:** The customer must be logged into their account.
- **Postconditions:** The support request is logged and acknowledged.
- **Main Flow:** Customer navigates to support section of app or website; fills out support request form detailing issue; Support Service logs request in database; service assigns a support ticket and sends acknowledgment to customer; support representative contacts customer to resolve issue.

Use Case 10: Wishlist Management

- **Description:** Customers can create and manage a wishlist of desired products.
- **Actors:** Customer, Wishlist Service
- **Preconditions:** The customer must be logged in.
- **Postconditions:** The wishlist is created or updated with the selected items.
- **Main Flow:** Customer browses products and selects items to add to their wishlist; Wishlist Service adds selected items to customer's wishlist in database; customer navigates to wishlist to view added items; service displays wishlist with product details. Can remove items or move them to shopping cart.

Implementation Steps & Matrix

Phase	Activity	Outcome
Analysis	Go through all use cases given in business requirements document (BRD) and understand well. If any clarifications are needed, connect with TLP and get them clarified. Document all assumptions that are not explicitly given in BRD that you would make use of them while implementing.	Document all clarifications and received assumptions on requirements.
Design	Based on your understanding of use cases, come up with high level functional modules, e.g. Customer, Payment Service etc., and assign use cases to those modules. To start with, assign a microservice to each high level module, e.g. Customer -> CustomerService, Payment -> PaymentService etc. Recap all microservices design principles and apply them to each service that you have come up in the above step and validate them. Based on the analysis, split a service into multiple services or merge some services into a single service if needed.	Mapping table of high level functional modules and use cases. Update above table with assigned microservices. Services interaction diagram.
Coding & Unit Testing	Implement controller, service and repository layers, entities and exception handlers and helper classes if needed. Configure Config Server, Eureka Server and API Gateway and integrate with microservice. Write unit test cases for controller, service and repository layers, ensure all test cases passed. Repeat above 3 steps for all microservices. Integrate distributed tracing (Micrometer), ELK stack, circuit breaker (Resilience4j), and Kafka based on requirement.	All microservices implemented end-to-end with following features integrated: - Config Server - Eureka Server - API Gateway - Micrometer - ELK stack - Resilience4j - Kafka
Deployment	Create docker images for all above microservices. Create docker compose file for one service completely, i.e., microservice, database, config server, eureka server, api gateway. Run docker compose file and test the microservice functionality if those are working correctly. Update docker compose file for remaining microservices and test if those functionalities are working correctly.	All microservices deployed as docker containers and working properly.

5. Non-Functional Requirements

This section outlines the non-functional requirements for this application utilizing a microservices architecture implemented with Spring Cloud, MySQL, OAuth2, and JWT. These requirements are essential for ensuring that the application meets performance, security, and operational standards.

5.1. Scalability

- **Horizontal Scaling:** Each microservice must be designed to scale horizontally by deploying multiple instances based on demand. Load balancing must be implemented to distribute traffic evenly.
- **Service Discovery:** Utilize Spring Cloud Netflix Eureka for dynamic service discovery, enabling microservices to find and communicate with each other efficiently.

5.2. Performance

- **Response Time:** The system should respond to user requests within 200 milliseconds for 95% of operations. For batch processing tasks (e.g., bulk transactions), the maximum acceptable response time should not exceed 2 seconds.
- **Throughput:** Each microservice should handle a minimum of 100 transactions per second under normal load conditions.

5.3. Security

- **Data Encryption:** Sensitive data (e.g., user credentials, personal information) must be encrypted in MySQL using AES-256 encryption. All communication between microservices must be encrypted using TLS (Transport Layer Security).
- **Authentication and Authorization:** Implement OAuth2 for user authentication, utilizing JWTs (JSON Web Tokens) for secure token-based access. Tokens must have an expiration time of 15 minutes and should be refreshable without re-authentication for active sessions.

5.4. Availability

- **Uptime Requirement:** The system must achieve 99.9% availability, with scheduled maintenance windows communicated in advance to users.

5.5. Failover Mechanisms

- **Circuit Breakers:** Implement circuit breaker patterns using Spring Cloud Resilience4j to prevent cascading failures and ensure resilience under load.
- **Data Protection & Compliance:** Ensure compliance with relevant data protection regulations such as GDPR and PCI DSS.
- **Audit Logging:** Maintain detailed logs of user activities, access events, and system changes for audit purposes. Logs should be retained for a minimum of 6 months.

5.6. Maintainability & Reliability

- **Code Modularity:** Each microservice must be modular and follow the Single Responsibility Principle, making it easier to update and deploy independently.
- **Configuration:** Each microservice must be configure all external properties using Spring Cloud Config server.
- **CI/CD Pipeline:** Establish a Continuous Integration/Continuous Deployment (CI/CD) pipeline using tools like Jenkins or GitLab CI for automated testing and deployment of microservices.
- **Error Handling:** Each microservice should have robust error handling and return appropriate HTTP status codes (e.g., 4xx for client errors, 5xx for server errors).

5.7. Architecture & API Gateway

- **API Gateway:** Utilize Spring Cloud Gateway to provide a single entry point for all microservices, enabling routing, load balancing, and additional security measures.

6. Conclusion

The Retail Microservices System aims to enhance operational efficiency and customer satisfaction through the outlined use cases. This architecture will allow for rapid deployment, scalability, and a superior user experience.